Journal of Systems Architecture 176 (2026) 103809

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

CREATOR-Sail: A RISC-V web simulator based on Sail ISA specification

Juan Carlos Cano-Resa, Felix Garcia-Carballeira *, Diego Camarmas-Alonso, Alejandro Calderon-Mateos

Universidad Carlos III de Madrid (ROR: https://ror.org/03ths8210), Departamento de Informática, Avenida de la Universidad, 30 (edificio Sabatini), Leganés, 28911, Madrid, Spain

ARTICLE INFO

Keywords:
RISC-V
Sail
Web simulator
Extensible simulator
Instruction set architecture

ABSTRACT

This article introduces CREATOR-Sail, a RISC-V web simulator that uses Sail as the instruction specification language. This enables binaries written in assembly language to be compiled, executed and debugged while simulating the complete RISC-V architecture and instruction set. The simulator can now simulate both 32-bit and 64-bit RISC-V variants and supports the simulation of the complete instruction set specified in the official standard, including vector and privileged instructions. Both variants include a cache memory module to increase the capacity for architecture simulation, mimicking a real processor. The aim of this work is to provide an industrial-level solution for companies and research groups, offering them a tool with which to customize the environment for their own purposes. The simulator includes an assembly compiler and a debugging module, making it more user-friendly and intuitive than other simulators. All simulator components have been integrated into the web environment using WebAssembly, which enables the execution of native code in a web environment and provides an execution performance similar to that of native applications compared to applications implemented only in JavaScript. The source code of web simulator is available at https://github.com/creatorsim/creator. The web simulator is available at https://creatorsim.github.io/creator.

Contents

* Corresponding author.
E-mail addresses: jucanor@pa.uc3m.es (J.C. Cano-Resa), felix.garcia@uc3m.es (F. Garcia-Carballeira), dcamarma@inf.uc3m.es (D. Camarmas-Alonso), acaldero@inf.uc3m.es (A. Calderon-Mateos).

https://doi.org/10.1016/j.sysarc.2026.103809
Received 16 February 2026; Received in revised form 16 April 2026; Accepted 16 April 2026
Available online 19 April 2026

1. Introduction

In development environments, the ideal scenario would be to test different instruction set architectures on a real processor. This enables developers to trial various configurations and implementations of each instruction set. However, the associated costs make this an unfeasible option for companies with limited budgets. For this reason, many companies opt for cheaper alternatives that produce the same results.

Against this backdrop, instruction set simulators have been developed, particularly for RISC-V architectures. Compared to ARM and x86, this architecture has experienced a 25% annual rise in use. The main reason for this is that RISC-V is modular and open source [1]. Even MIPS is transitioning to RISC-V [2].

In professional environments, instruction set architecture simulators are primarily used to define which architecture will execute the resulting product in a project. In research environments, they are used to develop new types of architectures or instructions that expand the capabilities of a specific architecture.

Instruction set architecture simulators are usually native applications that run on a computer. Examples include *QEMU* [3] and *Spike* [4]. However, they can also be run in a web environment, as demonstrated by CREATOR [5].

However, native applications require a lengthy installation and configuration process before they can be used. This makes this type of solution less appealing than a web application. The main reason for this is that users are inexperienced in using tools to code and simulate the behavior of an instruction set architecture. Additionally, the implementations of *QEMU* (coded in C) and *Spike* (coded in C/C++) lack a formal verification process for both the definition and behavior of the instructions and the emulated architecture. Consequently, this can lead to undefined behaviors and inconsistencies during execution. Furthermore, the use of native simulators is limited to the command line. For inexperienced users, working with the tool in this way does not generate interest in this type of architecture due to its complexity and the absence of an interface where components can be viewed.

CREATOR meets these requirements for native applications by providing a web simulator. A web simulator can be used on any platform with an internet connection and a web browser, such as computers, smartphones and tablets. This means that there is no need to install and configure the environment beforehand. It is generic and supports simulation across different architectures, such as MIPS [6] and RISC-V [7]. Additionally, users can edit the instruction set and simulated architecture using an interactive debugging interface, which makes the tool more accessible to inexperienced users. This feature indirectly enhances the simulator's appeal. All of this must be considered alongside the fact that CREATOR was not initially intended for use in industrial environments, with features such as adapting the architecture to the developer's needs or conducting more in-depth testing of the capabilities of the RISC-V architecture.

The aim of CREATOR [5] is to design a complete web simulator for the RISC-V architecture. To this end, a new RISC-V simulator has been developed based on the RISC-V standard specification and implemented in Sail. This work expands the instruction set architecture and its variants to those specified in the RISC-V standard [8,9]. Additionally, a cache architecture module has been added to simulate the behavior of a real processor in the RISC-V architecture. To this end, the currently implemented instruction set (32-bit architecture and IMFD instruction set) has been extended to cover the entire RISC-V standard specification.

During development, Sail [10] was used as the specification language for both the instruction set and the architecture definition. This is because simulators were traditionally specified in natural-language documents, which resulted in a large amount of ambiguity and inconsistency. While the situation improved over the years with the use of high-level code languages, they did not provide formal verification of the architecture's definition and behavior. However, Sail solved this problem by providing a specification language for instruction-set architectures that formally verifies the definition and behavior of the simulated hardware and the instruction set architecture that was designed and implemented. This also allows us to implement a cache architecture that has been included in the simulator.

The rest of the document is structured as follows: Section 2 describes the state of the art. Section 3 presents a new web simulator that expands the current version of CREATOR, offering developers a solution to help them understand the different functionalities of the new simulator tool. Section 4 shows how the architecture and instruction set implemented in the simulator are validated. Section 5 reviews some use cases to demonstrate the different workflows available in the simulator. Section 6 describes conclusions and future work.

2. Related work

The needs of professional and research environments outlined above have led to the development of multiple instruction set simulators, especially those based on RISC-V. Those based on RISC-V have become particularly popular in recent years, with a 25% annual increase in use, due to their presentation as an open-source, modular model, as opposed to standardized architectures such as x86 and ARM [1]. Next, this paper provides a review of the most widely used simulators in educational and professional settings, based on the popularity of these tools and the interest shown by the community. The simulators that will be analyzed are QEMU, Spike, Renode, RARS, Ripes, Venus and CREATOR.

QEMU [3] is an open-source simulator and virtualizer that runs as a native application on a computer. This means that it needs to be installed and configured before it can be used. It can be used in various ways, including as a virtual machine, a tool for application deployment (such as Docker) or an instruction set architecture simulator. This paper focuses on the last use case. The architecture and instruction set implementation is coded in C, and the simulator can simulate various architectures (such as MIPS, RISC-V and ARM). The debugging process must be performed through *GDB* [11]. One strength of this simulator is that it does not require a kernel layer to simulate the architecture. However, QEMU does not have an implementation of a cache memory module. The user can, however, apply one from the simulator's implementation. The tool makes it possible to run an assembly program that has been precompiled for the desired architecture. However, as the simulator lacks a user interface layer, interactivity is limited to the operating system's command line. Consequently, the simulator presents difficulties for users who are inexperienced with this type of solution. This is related to the expensive installation and configuration process, which can increase the learning curve associated with the architecture and use of the simulator. These reasons, in addition to the difficulty of editing and customizing the architecture (changing the behavior of the instructions or inserting new instructions) for each user's specific needs, the size of the source code and its structure (it contains many architectures and sets of instructions to simulate), and the fact that the whole simulator must be rebuilt for every change made (no matter how small or large the change), make it an unviable solution for these development environments.

Like QEMU, Spike [4] is an instruction set simulator that runs as a native application in the operating system's command line. It exclusively simulates the RISC-V architecture, is open source and was developed by the RISC-V organization. Like the previous simulator, it allows the modification of the architecture and instruction set. However, it only contains the RISC-V architecture, making it easier to customize these elements. Additionally, the simulator has an integrated debug module, but it can also utilize a debug module integrated into the operating system, such as *GDB*. However, as with the previous simulator, Spike does not have cache memory implementation. To run a program on the simulator, a minimum kernel layer must be applied; this can be downloaded from *RISC-V-pk* [17], which is implemented by the RISC-V organization. Using the simulator is similar to using QEMU.

Table 1
Comparison of the most widely used RISC-V simulators in research and educational environments.

Simulator	QEMU [3]	SPIKE [4]	Renode [12]	RARS [13]	Ripes [14]	Venus [15]	CREATOR [16]	CREATOR-Sail
Implementation	C	C/C++	Dart	Java	C++	Java + JavaScript	JavaScript	JavaScript + Wasm
Graphic user interface	✗	✗	✗	✓	✓	✓	✓	✓
Architectures to simulate	Multiple	RISC-V 32	Multiple	RISC-V 32	RISC-V 32	RISC-V 32	Multiple	RISC-V (32/64)
Cache memory module	✗	✗	✗	✗	✓	✗	✗	✓
Extensible	✓ (hard to edit)	✓ (hard to edit)	✓	✓(hard to edit)	✓ (hard to edit)	✗	✓	✓
Debug mode	✓	✓	✓	✓	✓	✓	✓	✓
Installation required	✓	✓	✓	✓	✓	✗	✗	✗
IDE	✗	✗	✗	✓	✓	✓	✓	✓ (multi-file)
Command line integrated	✓	✓	✓	✓	✗	✓(limited)	✓	✓
Purpose	Professional	Professional/Educational	Professional	Educational	Educational	Educational	Educational	Professional
RISC-V ISA	Complete	Complete	Almost Complete	IFMD	IM	IM	IMFD	Complete

It allows the execution of assembly programs that have been compiled using an external tool, and interactivity is limited to the command line. In other words, it does not have a graphical user interface.

It should be noted that these simulators cannot be used with assembly code as direct input, as they lack an integrated compiler. Therefore, it is necessary to use an external tool to compile a program that matches the simulator's input. While these simulations may be viable in professional environments, they do not provide clear information about what is happening during execution, making it difficult to understand their behavior.

Renode [12] is a framework used for simulating complete embedded systems. It does not just execute CPU instructions; rather, it is geared toward simulating entire platforms comprising CPUs, peripherals and buses. It stands out for its full integration with RISC-V and its status as a cross-platform simulator. In addition to its use cases, it allows the simulation of operating systems on this architecture. However, its implementation is not adapted to the standard used in RISC-V, and since it is a native simulator, it requires a prior installation process; there is also a learning curve for its use.

The RARS simulator [13] is used in educational contexts to teach assembly language and the RISC-V architecture. It was developed to complement the MARS simulator [18], which is designed for the MIPS architecture. However, it does not include the full set of RISC-V instructions, it runs locally and it lacks a user-friendly interface.

Ripes [14] is an educational simulator used to teach assembly language and the RISC-V architecture. It visualizes the internal functionality of a processor and demonstrates pipeline execution, forwarding and stalls. While there is a version that can be run in web environments, this version does not include the full RISC-V instruction set and is not compliant with the RISC-V instruction definition standard.

Venus [15] is a well-known RISC-V educational simulator that is widely used in educational settings to teach assembly language and debug programs. Although it has a web-based version, this version does not include the full RISC-V instruction set and only simulates the 32-bit architecture variant.

Other simulators that are also classified as RISC-V simulators by the RISC-V organization focus on different aspects of interest to users. *QtRvSim* [19] is an online simulator used in educational settings to learn assembly language and the RISC-V architecture; *jor1k* [20] is a simulator that allows users to demonstrate how to run an operating system on different architectures, such as RISC-V, from a web-based environment; *riscv-rust* [21] is an educational simulator designed to simulate an operating system based on the RISC-V architecture in a web environment, helping users to understand the behavior and communication of the various elements of the architecture; and *Vulcan* [22] is an educational simulator designed for learning and debugging instructions executed on a RISC-V architecture.

While these simulators all offer interesting features related to the RISC-V architecture in academic and research areas, they are not comprehensive solutions capable of simulating the full set of RISC-V instructions. They also lack a clear and intuitive user interface, which would make them more user-friendly. Furthermore, none of the simulators presented thus far have undergone any formal verification process, such as that established by RISC-V for developing its architecture using Sail, coupled with a robust suite of validation tests.

Finally, CREATOR [5] is a web-based simulator[1] that can be accessed from any device with a web browser installed. It is multiplatform and uses generic assembly code to simulate RISC-V and MIPS architectures. Implemented in JavaScript, it features a graphical user interface, an IDE, a compilation tool and an execution module, all of which are integrated within the simulator. This provides greater ease of use compared to other simulators on the market, as it is perfectly adapted to the user experience with this type of tool. Notable features include the ability to customize the architecture and instruction set, a simple and visual debugging module to facilitate this process and a graphical, interactive map of the architecture status that is updated during the execution of each instruction. It is also worth noting that CREATOR supports the RISC-V IMFD instruction set but does not have cache memory implementation.

Table 1 shows a comparison of actual simulators designed for RISC-V in educational, professional and research environments. This presents a new opportunity for a web-based RISC-V simulator. For this reason, we propose a new web simulator to complement the CREATOR implementation. The instruction set and emulable architectures have been expanded, and a cache memory module has also been added to the architecture. The aim is to provide professional development and research environments interested in this architecture with a comprehensive web simulator. A notable feature of the new simulator is that the architecture and instruction set specifications have been modeled using Sail [10] as the semantic specification language of the instruction set. This language enables the formal verification of the emulated architecture's definition and behavior, as well as the instruction set's implementation, throughout the process. Further details about the language can be found below.

3. CREATOR-Sail

In this section, we will first address the design of the simulator tool's new architecture and explain how it is developed and integrated with the backend and frontend. Next, we will present the new web simulator integrations and demonstrate the different functionalities through a use case example.

These new integrations include the implementation of a new simulation component with Sail, which covers the entire instruction set specified in the 32-bit and 64-bit RISC-V variants. There is also a new assembly compiler tool for RISC-V programs that generates executable files for both variants of the simulator. Finally, the simulator's web interface has been updated to include new functionalities.

[1] Available at https://creatorsim.github.io/creator

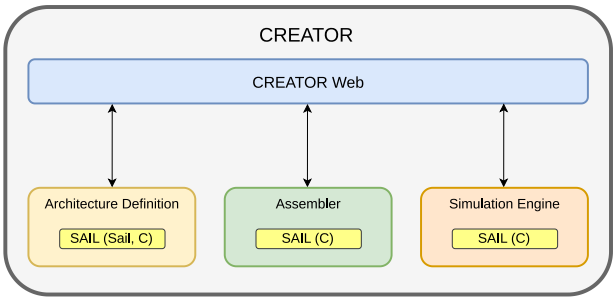

Fig. 1. Design of CREATOR-Sail architecture components.

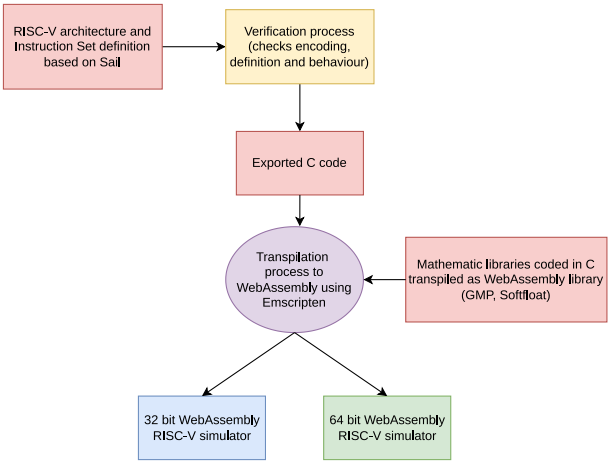

Fig. 2. Workflow for developing a new Sail-based RISC-V architecture simulation engine.

3.1. Design of the solution

This subsection describes the design of CREATOR. As shown in Fig. 1, it has a modular structure comprising several components. The first layer of CREATOR's software architecture corresponds to the user interface, comprising a web-based graphical interface developed using responsive web technologies such as Bootstrap 5 [23] and Vue 3 [24]. This layer uses CREATOR's internal modules to define the architecture and assembly programs that use it (compiler engine), as well as to simulate their execution (simulation engine). Thanks to WebAssembly [25], these components have been integrated into web environments, enabling the execution of binary code in web browsers with a performance comparable to that of a native application [26]. JavaScript also integrates the WebAssembly module into web environments, making it easier to filter the output generated by WebAssembly and connect it with the different components of the user interface.

The architecture definition module is based on Sail and C, and it describes the architecture in terms of the various elements that comprise it, their corresponding register sizes and the available instruction set. The RISC-V program compiler for the Sail-based simulator contains the full set of instructions in the RISC-V ISA, enabling the generation of assembly language programs. Finally, the simulation engine allows the RISC-V architecture to be simulated and programs generated by the compiler to be executed.

The simulation engine was developed based on the Sail architecture and instruction set implemented by the RISC-V organization. This was then exported to C for transpilation to WebAssembly, as shown in Fig. 2. It should be noted that the code of the C libraries required for binary generation (GMP, an arithmetic precision library, and Softfloat, a floating-point operation library) also needed to be transpiled. Once all

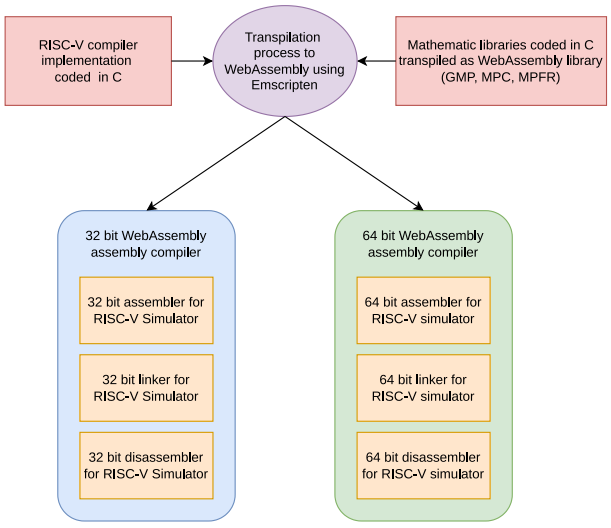

Fig. 3. Workflow for developing the new RISC-V assembly compiler.

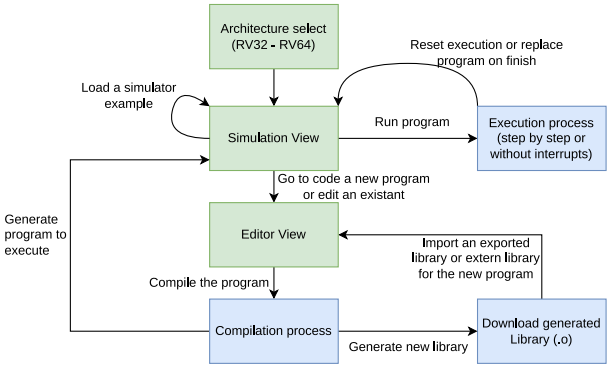

Fig. 4. Diagram illustrating the workflow of the CREATOR execution: architecture selection, program edition, compilation and program execution.

parts of the simulator are independent of the operating system during the simulation generation process, the entire simulator is transpiled using Emscripten [27], thereby generating both the 32-bit and 64-bit RISC-V architecture simulators in WebAssembly.

Meanwhile, the development of the new compiler has relied on the compilation tool implemented by the RISC-V organization and written in C. As shown in Fig. 3, the necessary compiler generation libraries are transpiled to WebAssembly to avoid operating system dependencies when generating the new compiler (GMP for arithmetic precision, MPC for complex numbers and MPFR for floating-point precision).

Once the various components of the compiler have been adapted for each architecture variant, they are transpiled to WebAssembly using Emscripten [27] and integrated into the user interface using JavaScript.

Finally, the CREATOR-Sail architecture components are integrated into the web simulator using JavaScript. This is required by WebAssembly in order to load the module responsible for compiling and executing programs written in assembly language. This is illustrated in Fig. 4. This diagram illustrates the new execution flow within the simulator.

First, when an architecture variant is selected, the simulator loads the necessary modules for compiling assembly programs on that architecture asynchronously. Once the program has been compiled, the compiler generates the corresponding binary to be launched in the executor module or the library, if the program is intended for a future project. Next, the execution module for the selected architecture is loaded asynchronously once the binary is generated. During simulation, the WebAssembly program and user interface communicate constantly

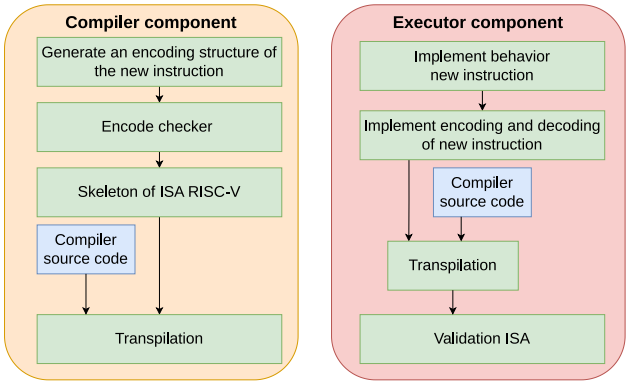

Fig. 5. User workflow for editing the architecture and/or instruction set.

to report the current execution state and display it in the different architecture modules represented in the user interface. These include the interrupts generated by program execution (ecall) and the interrupts created by the user during execution (breakpoints). Once execution has ended, either through user intervention or otherwise, the various WebAssembly modules can be restored to their initial loaded state, either for a new complete flow or for executing only the program in the executor.

Another way to use CREATOR is to adapt the new components to the user's needs by editing existing instructions, adding new ones, and integrating new architectural components. To do this, the user must follow the steps for each relevant component, as shown in Fig. 5.

First, the user needs to define the structure of the new instruction they want to introduce and check whether any other instruction with the same structure is included in the same instruction set in the extension. Once this has been done, the user must integrate the structure and encoding into the compiler source code in order to generate a new version of the compiler. This is because the compiler needs the structure and encoding format when generating the RISC-V program to run on the executor module. Once the new compiler version has been generated, the user needs to implement the behavior of the new instruction, along with its encoding and decoding formats, in Sail. During program execution, the simulator fetches and decodes the new instruction, determining which instruction is being referenced. Finally, both components are validated to ensure compliance with the ISA defined in the standard.

3.2. Simulation engine

Traditionally, the design and implementation of architectures were described in documents written in natural language. However, this approach led to ambiguities and problems during verification. This situation evolved into the use of high-level code languages such as C or Python, which allow us to prove the design of the architecture. However, these languages still lack a formal verification process to check semantic correctness, instruction behavior, security against weaknesses and so on.

Given this need to formally verify the design of an architecture, a language that allows such verification was developed: Sail. The RISC-V organization adopted this language to design and verify the instruction set architecture that they were developing. Verification helps to avoid errors such as Intel's $FDIV$, which cost 500 million dollars in 1995 [28].

Sail is a specification language for instruction set architectures. Its behavior is similar to that of a framework. It gives developers the ability to implement instruction structures, their behavior, encoding and decoding, the involved hardware resources and the processor architecture. The main feature of this language is that it is not compilable,

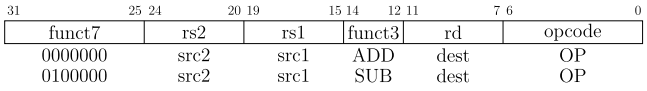

Fig. 6. Diagram of encoding structure for an instruction defined in the RISC-V standard.

meaning that it does not allow the generation of a simulator that can be executed. In Sail, the syntax, behavior and data type are checked to ensure that they are free from ambiguity or inconsistency during execution.

Sail allows the specification to be exported to a high-level language for simulation (C or OCaml) and the simulator to be generated. It also enables mathematical models to be implemented and proven in tools such as *Isabelle* [29], *HOL4* [30] and *Coq* [31]. These tools enable the formal verification of the behavior of emulated software and hardware, as well as providing security against weaknesses that could be exploited by attackers, through mathematical and logical demonstrations.

This language has been used to specify the RISC-V instruction set architecture, based on the specification implemented by the RISC-V organization.[2] This definition was used as the basis for developing the new simulator. The repository contains the complete implementation of the RISC-V instruction set; some of these instructions have been adapted to meet the requirements of the project. System calls have been highlighted, and while maintaining the standard definition, they have been updated for integration into the simulator web. This includes a simple and intuitive debugging mode to facilitate this task.

The main reason for choosing this solution is that while JavaScript is not a suitable language for running applications in web environments, WebAssembly is. It improves the execution performance by an average of 30% and offers better management of browser resources [32]. This tool is intended to complement JavaScript, not replace it, as JavaScript is required to execute its modules. However, WebAssembly offers a better execution performance for native applications in web environments, achieving a performance close to that of a native application in some situations, as cited in [26].

Regarding the simulator and the language used to specify the RISC-V architecture and instruction set, the specification was made using Sail. The basis for the architecture and instruction set specification is an extracted RISC-V instruction (add rd, rs1, rs2) from the repository. This instruction set specification is attached.

First, it is necessary to explain how instructions are encoded. Each instruction contains a code that specifies how it should behave when executed and which elements are affected.

As shown in Fig. 6, the operation code generally consists of two or three fields. These fields specify the type of instruction (operations with registers, branch operations, operations with immediate values, etc.) and the exact operation to be performed (addition, subtraction, bit shifting with registers, etc.). Specifying the type and behavior of an instruction requires two fields. However, if more fields are required to represent and identify additional instructions of a given type due to size limitations, an extra field is added to the code to compensate for the lack of values. Each instruction has a unique operation code to differentiate it from others and avoid ambiguities during execution. The remaining fields in the instruction relate to the resources required for execution. These fields may contain registers, immediate values or absolute or relative memory addresses.

The behavior specification of an instruction with Sail syntax is shown in Fig. 7.

This fragment of code first defines the operation inside a type of instruction (*rop*), and with this, the structure of this type of instruction

[2] Available at https://github.com/riscv/sail-riscv

```
enum rop = {RISCV_ADD, RISCV_SUB, RISCV_SLL,
            RISCV_SLT, RISCV_SLTU, RISCV_XOR,
            RISCV_SRL, RISCV_SRA, RISCV_OR,
            RISCV_AND}

union clause ast = RTYPE : (regidx, regidx, regidx, rop)

mapping clause encdec = RTYPE(rs2, rs1, rd, RISCV_ADD) <->
    0b0000000 @ rs2 @ rs1 @ 0b000 @ rd @ 0b0110011
mapping clause encdec = RTYPE(rs2, rs1, rd, RISCV_SUB) <->
    0b0100000 @ rs2 @ rs1 @ 0b000 @ rd @ 0b0110011
```

Fig. 7. Definition of the type, and encoding of the structure and fields of the instructions to be implemented.

```
function clause execute (RTYPE(rs2, rs1, rd, op)) = {
  let rs1_val = X(rs1);
  let rs2_val = X(rs2);
  let result : xlenbits = match op {
    RISCV_ADD => rs1_val + rs2_val,
    RISCV_SUB => rs1_val - rs2_val
  };
  X(rd) = result;
  RETIRE_SUCCESS
}
```

Fig. 8. Implementation of the behavior of instructions following their definition. Example of a RTYPE instruction (add, sub, etc.).

```
mapping rtype_mnemonic : rop <-> string = {
  RISCV_ADD <-> "add",
  RISCV_SUB <-> "sub"
}
mapping clause assembly = RTYPE(rs2, rs1, rd, op)
  <-> rtype_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
      reg_name(rs1) ^ sep() ^ reg_name(rs2)
```

Fig. 9. Assembly and disassembly of the defined instruction to specify the fields of the instruction to the simulator.

(*RTYPE*) can be defined. Specifically, *RTYPE* instructions are register instructions, so to execute them it is necessary to identify three registers (*regidx*): two as operands and one to store the result of the operation. The operation code (*rop*) indicates how the instruction should behave. For simplicity, this can be likened to an abstract syntax tree (AST).

In order to differentiate between instructions of the same type and make them identifiable by the simulator, a mapping is established between decoded and coded functions. This makes it easier for the simulator to execute instructions that arrive in binary code.

Once the simulator can decode the instruction and identify the corresponding values, it can implement the behavior of different *RTYPE* instructions depending on the value contained in *rop*. First, the values to be operated on are extracted from the registers (*rs1, rs2*). Then, the operation is identified (*match op*) and executed, and the result is stored in the destination register (*rd*), as can be seen in Fig. 8.

Finally, to improve the completeness and verification of the instruction specification, the relationships between disassembled instructions (e.g., add, sub) and assembled instructions are defined, as shown in Fig. 9. This process is then repeated for the remainder of the RISC-V instruction set, as well as for the definition of the processor architecture's simulated hardware resources.

Once the architecture has been implemented and verified, it is exported to C to build the web simulator, which will then be integrated into CREATOR. During the development process, a minimal kernel layer was implemented. The main objective of this is to provide development support and make the simulator easier to use. However, it is possible to implement a custom kernel for personalized management and adaptation to the developer's needs using assembly language from the CREATOR interface itself. Additionally, Sail enables users to enter new instructions or modify existing ones. After modification, the implementation and behavior designed prior to simulator generation are robustly verified.

This simulator is generated with WebAssembly from the implementation exported from Sail to C, as it supports generating executable applications in web environments with the same performance as a native application and without the need for major modification to the implementation, as mentioned above [26].

3.3. Compiler engine

The CREATOR compilation process was similar to a filter that checked whether the instructions were well coded. However, this compiler did not generate an executable binary through compilation. Instead, the simulator dynamically translated the received instructions and, depending on their type and values, emulated their execution. This situation changed with the integration of a compilation tool that generates executable binaries on RISC-V architectures. The main reason for this is that increasing the size of the instruction set to be executed by a significant amount can negatively affect JavaScript performance.

To this end, the tool developed by RISC-V has been used to generate executable binaries on RISC-V architectures. This tool is a toolchain[3] that allows programs to be generated from assembly code or C code, and it has been adapted to the needs of the simulator. Specifically, the code assembly, linking and disassembly tool for RISC-V has been used. It has also been integrated into the simulator using WebAssembly to run the native application in CREATOR. The compilation process now consists of three phases:

1. **Assembly:** Verify that the code has been implemented correctly and that there are no syntax or implementation errors in order to generate an object file (*.o*).
2. **Link:** Proceed to the link phase with a linker script (*.ld*) that contains a map of the regions and memory addresses where the binary is expected to run in the RISC-V simulator.
3. **Disassembly:** Once the executable binary has been generated, proceed to the disassembly phase, which involves translating the memory addresses assigned to the encoded data and instructions. This final step can make debugging the simulator's execution easier and provide a better understanding of how it behaves during execution.

Also, this new compiler enables users to introduce new Sail-defined instructions in any RISC-V extension. This new compiler generates files that can be downloaded and used on real RISC-V processors.

3.4. Cache memory module

The previous version of CREATOR did not have a cache memory module. Therefore, we have included one in this new version. This module may increase the tool's simulation capacity. The purpose of the cache simulator is to analyze the hit/miss rate generated by the execution of an assembly program, therefore, it does not focus on issues related to execution cycles or execution times.

The cache memory module is based on two components (see Fig. 10):

- Cache architecture definition module that allows you to configure the cache's hardware settings.
- Hit/miss ratio analyzer that analyzes the hit and miss rates based on memory accesses generated by the simulation engine.

The cache architecture definition module enables you to configure up to two cache levels: L1 and L2. Each level can consist of a unified cache or split caches for instructions and data. You can also configure the following parameters: the number of lines in each simulated cache; the size of the lines; the cache mapping policy (direct mapping,

[3] Available at https://github.com/riscv-collab/riscv-gnu-toolchain

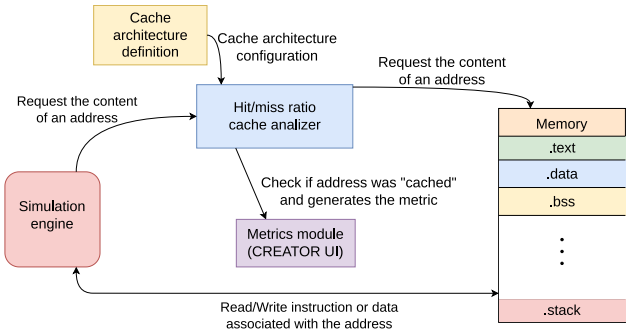

Fig. 10. Diagram of the cache simulation module and its integration with the simulation engine.

fully associative, and set-associative); and the replacement policy. This module enables users to design their own cache memory model, setting the number of lines and block size for each cache independently. This enables users to explore the cache memory model fully and determine the most suitable configuration for their project.

The hit/miss ratio analyzer captures the memory addresses of the data and instructions generated by the simulation engine. It then virtually verifies the hit or miss of the generated reference based on the cache architecture definition, because the cache does not store data and its sole purpose is analysis. The result is communicated to the web interface, which updates the metrics. Execution then continues by requesting stored data or instructions from memory. Finally, in read operations, the contents of the memory are sent directly to the simulation engine, and in write operations, they are sent directly from the simulation engine to memory.

It should be noted that this cache model is entirely simulated. This means that the actual content of the memory is not stored in the cache; only the addresses of the accessed memory are stored. Real data storage has not been implemented mainly because the simulator's main memory model does not reserve the total capacity of the architecture's address space. In other words, the simulator dynamically reserves the memory addresses being used while the program is executing. For example, if a program allocates different values to different labels, the simulator allocates the associated data and instructions in memory when it loads the binary. However, the rest of the memory addresses are not allocated. If an instruction writes to an unallocated address, the simulator dynamically allocates it for use. Therefore, if a robust cache memory model were implemented based on the simulator's main memory model, execution would be severely slowed down because unnecessary memory allocations would be made dynamically, and most of them would not be used outside of the single access to the cache that would be made during program execution.

In summary, the cache memory model implemented in the new version of CREATOR has different cache mapping policies (direct mapping, fully associative, and set-associative), with up to two levels (grouped or separated) and with a block size in each independent cache; the available replacement policies are currently FIFO and Random. However, it is expected that replacement policies will be expanded in the future to provide greater simulation capacity at the cache memory level.

3.5. User interface and web environment

Previously, the implementation of CREATOR did not contain the different functionalities that have been implemented during project development, but its evolution positions it as a much more comprehensive web simulator compared to other development tools and platforms analyzed in the related work section. Some notable aspects of its evolution include the simulation of different architectures, the page's interactivity, the ability to use predesigned libraries and the register

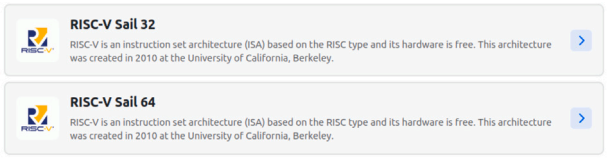

Fig. 11. CREATOR home page that displays new Sail-based architectures.

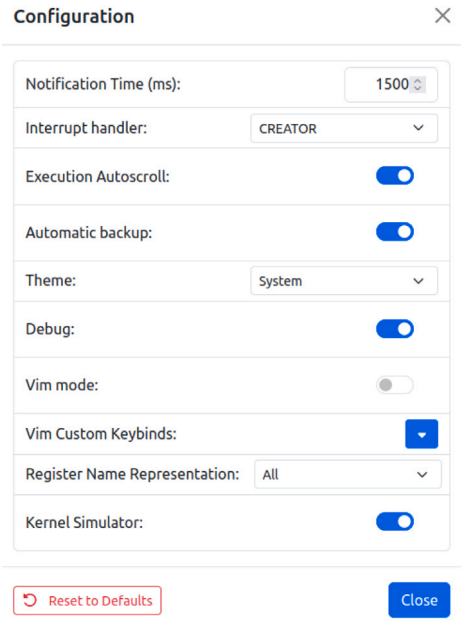

Fig. 12. CREATOR-Sail configuration menu module where the user can edit or modify some characteristics of the simulator interface.

and memory map that enables users to watch the behavior during execution. However, the aspect that makes it stand out from other simulators is that it does not require any previous installation process for use, and it is multi-platform.

The previous version of CREATOR supported a 32-bit architecture with a reduced set of instructions (*IMFD*). While this simulator is useful for small projects and teaching environments, the trend of many processors being designed to support a 64-bit architecture, coupled with the new integrations presented in previous sections, led to the aim being to offer professional developers and researchers a complete solution in this area.

The new integrations in CREATOR have led to changes in the user interface and its use, as well as in the behavior of the features already implemented in the web simulator.

Both the 32-bit simulator and the new 64-bit simulator for the RISC-V architecture are now available with an implementation based on the Sail definition, as shown in Fig. 11.

In addition, a module for selecting instruction set extensions has been added, as shown at the bottom of each architecture selection module in Fig. 11. This allows the user to configure the extensions that they want to work with in the simulator before selecting the architecture variant. However, if the user wants to work with a set of extensions that have not been selected on the home page, they can modify this configuration in the simulator in the configuration menu and customize the instruction set that they want to work with.

Within the configuration menu, the user can select whether they want to work with the kernel integrated into the simulator itself or with a kernel they have implemented, as shown in Fig. 12.

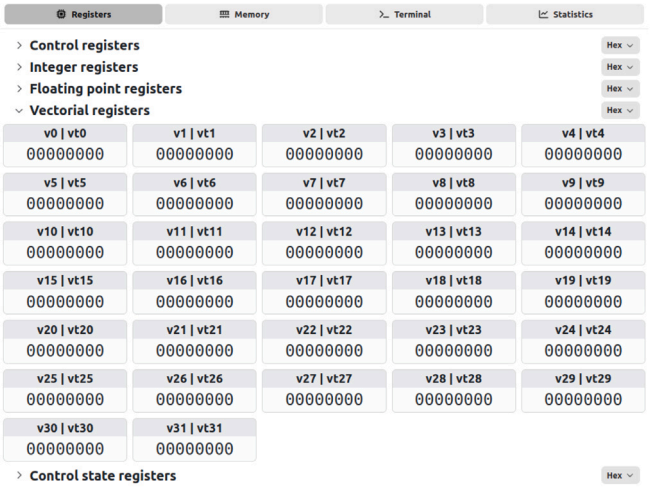

Fig. 13. Vector register file that displays the current state of registers during program execution.

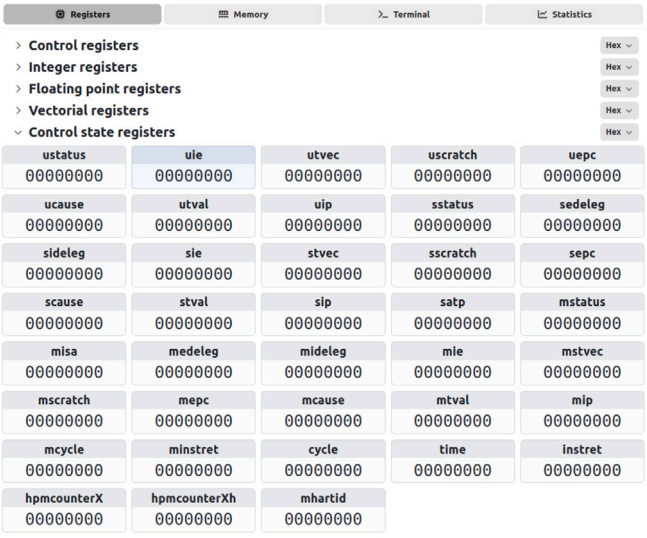

Fig. 14. Control-State Register file that displays the configuration and state of the control registers during the program execution.

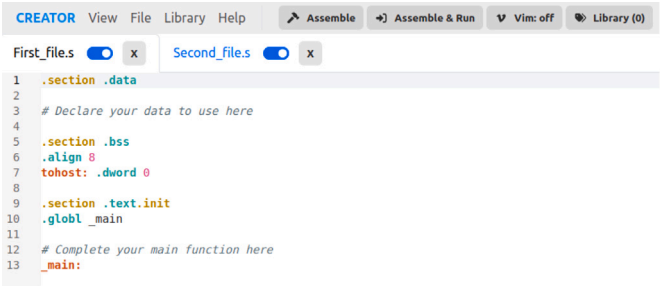

Fig. 15. CREATOR multi-file editor interface similar to a professional code editor.

In the architecture view, the interface has been updated to display the cache memory configuration that will be applied during execution. This configuration can be edited in this view by selecting which cache levels to use, the number of lines in each level, the number of lines in each cache module, the size of the cache block and the location and replacement policy.

As previously mentioned, the user can configure the cache architecture to be applied during the simulation. Each cache size ranges from 32 to 2048 lines for separate instruction and data caches, and from 32 to 4096 lines for caches that store both instructions and data. The user can also select the block size of each cache, ranging from 32 to 128 bits, as well as selection modules for the replacement and location policies of the cache memory model. Please note that the cache module can only be configured when there are no assembly programs running during the simulation. This is to avoid undefined behaviors.

In the simulation view, the interface has been updated to display all the elements of the architecture that can be interacted with in the simulator.

As can be seen in Fig. 13, a vector register file has been added to the existing register file module. This illustrates the changes to the architecture when instructions belonging to the vector extension are executed.

Additionally, the Control-State Register file has been added, as can be seen in Fig. 14. This aims to illustrate and explain the configuration of the architecture, and justify the behavior when instructions are executed.

This register file comprises registers that are updated according to execution permissions (user, supervisor and machine). These registers enable users to understand instruction execution behavior and, in the event of an exception being generated, identify the cause. Each register can be updated by executing instructions and with the appropriate permissions to prevent unwanted behavior and ensure the correct management of the architecture. This register file can also be very useful for debugging processes, justifying behavior during project development and learning about aspects of the simulator configuration.

Finally, the integrated web editor has been updated to behave like professional code editors such as VSCode and SublimeText, as shown in Fig. 15. This update is intended to make programming and organizing the project more convenient.

The code space module is similar to previous versions of CREATOR. However, the editor can now work with more than one file of assembly code and browse different files without losing the project's progress. The content of each file is stored automatically, eliminating the need for manual storage by the developer and making the simulator easier to use.

After developing the program in one or multiple files, it is necessary to indicate which ones the user wants to compile to generate the program that will run in the simulator. If no files are indicated or the contents of the files are empty, an error will be returned stating that a minimum amount of code is required to generate a program to run in the simulator.

Once the program has been compiled, the developer can run it directly in the simulator or download it as a library of subroutines for use in another project.

4. RISC-V ISA validation

This section will address the validation of the architecture and the instruction set based on the Sail language.

Although the Sail framework verifies and validates code during development and export to prevent undefined behavior during execution, additional validation is necessary to ensure that the simulator's behavior does not differ from that specified in the ISA. For this reason, the *riscv-tests*[4] repository has been used, as the RISC-V organization relies on these tests to verify that the architecture and instruction set execute correctly in accordance with the standard.

[4] More information is available at https://github.com/riscv-software-src/riscv-tests.

Table 2
Target environment configurations available to test architecture.

Target environment name	Description
p	No virtual memory, only core 0
pm	No virtual memory, all cores
pt	No virtual memory, timer interrupt
v	With virtual memory

Table 3
Battery test of RISC-V programs to validate RISC-V architecture.

TVM Name	Description
rv(32/64)ui	RV32/RV64 user-level, integer only
rv(32/64)si	RV32/RV64 supervisor-level, integer only
rv(32/64)ui	RV32/RV64 user-level, integer only
rv(32/64)uf	RV32/RV64 user-level, integer and floating-point
rv64uv	RV64 user-level, integer, floating-point and vector
rv(32/64)sv	RV32/RV64 supervisor-level, integer and vector

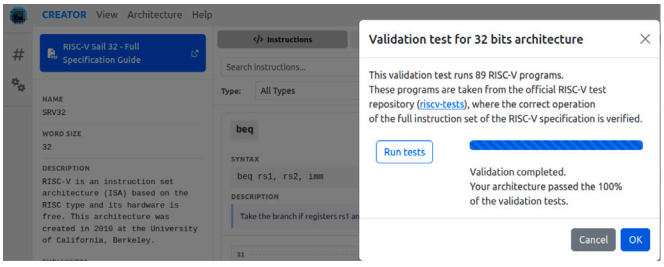

Fig. 16. Validation process of instruction set applied for the 32 RISC-V specification.

These tests cover the entire instruction set and can be run in a variety of environments, as shown in Table 2. For simulator validation, however, only the first environment is considered. This is because, in web environments, virtual memory scalability is limited by the web browser and multicore execution is not currently enabled. Furthermore, the objective of this validation process is to ensure that all instructions in the full RISC-V specification function correctly.

The tests defined in this repository cover all instructions and extensions formally specified in the RISC-V standard. As shown in Table 3 (extracted from the repository), the validation process covers all RISC-V instructions. Furthermore, these tests are run with all the data types and structures available in RISC-V. There are 89 tests available for execution on the 32-bit variant and 122 on the 64-bit variant. The 64-bit variant has more tests because its instruction set is larger than that of the 32-bit architecture.

Each time a component of the CREATOR architecture is modified, the architecture is validated at the implementation level and the simulator is checked to ensure that it has been implemented correctly before the WebAssembly program is generated. Additionally, in the web application's functional architecture, we have incorporated a new section into the simulator's description that enables the execution of all tests defined in the RISC-V test repository. This new functionality of the web simulator (Fig. 16) enables the correct execution of all defined test cases to be validated, thereby confirming the reliability of the developed simulator.

The next section will present two use cases to demonstrate the integration of the different modules and the available workflows in CREATOR.

5. Applications of the system based on the proposed design

This section presents two use cases that demonstrate the two available workflows in the simulator.

```
## Auxiliar progam code
    .section .text.init
    .globl _init_vec

    _init_vec:
        # Load the number of vectorial
        # elements to use in each
        # instruction
        lw t3, N

        # Configure vectorial length
        # and size of each element
        # (64 bits)
        vsetvli t5, t3, e64

        # Return execution to
        # the _main function
        jr ra
```

Fig. 17. Use case for vector instructions: auxiliary assembly program that configures the size of vectors.

```
## The main program code
    .section .data

    .align 3
      N: .dword 8
    .align 3
      vector_1:
        .dword 0, 1, 2, 3, 4, 5, 6, 7
    .align 3
      vector_2:
        .dword 10, 9, 8, 7, 6, 5, 4, 3
    .align 3
      vector_3: .space 64

    .section .bss
    .align 8
     tohost: .dword 0

    .section .text.init
    .globl _main

    _main:
        call _init_vec

        # Load the content of
        # each vector from memory
        la t0, vector_1
        vle64.v v1, (t0)
        la t1, vector_2
        vle64.v v2, (t1)

        # Add the content and
        # store the result
        vadd.vv v3, v1, v2
        la t2, vector_3
        vse64.v v3, (t2)

        # Exit the program
        li a0, 10
        ecall
```

Fig. 18. Use case for vector instructions: main assembly program that executes the addition of two 64-bit vectors in a 64-bit architecture.

5.1. Use case: Compile and execute a RISC-V 64-bit program using vector instructions

This use case involves coding and running a program that executes vector instructions in a 64-bit RISC-V architecture. The methodology to be followed during the use case is as shown in Fig. 4.

The program to illustrate this use case is defined in Figs. 17 and 18.

On the one hand, a file establishes the size of each element of the vector (64 bits per element), to be treated by the instruction that operates with vector registers. These files define an auxiliary function to be called by the main function, as shown in Fig. 17. On the other hand, the second file encodes the main functionality of the program. The main function loads two vectors with some values that have been

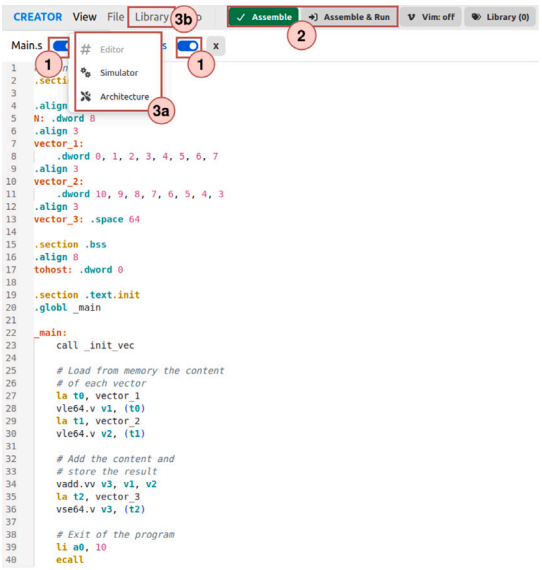

Fig. 19. User interface for editing and compiling programs implemented in assembly language.

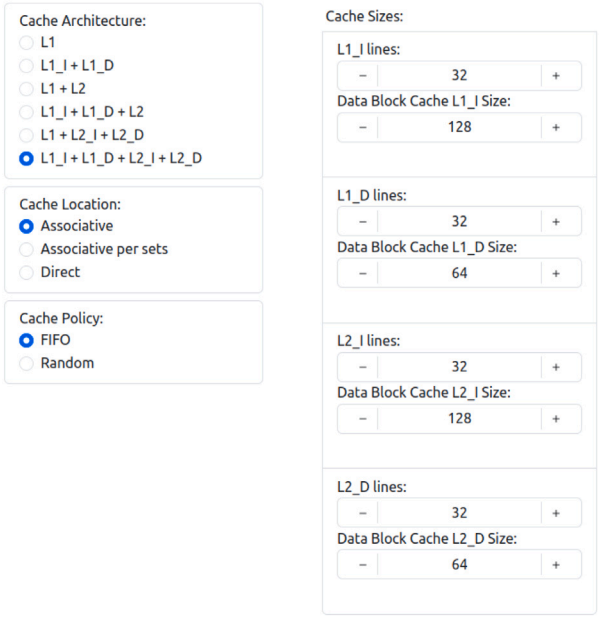

Fig. 20. Cache configuration available to be applied during execution.

declared in the data section (*.data*), adds them, and the result is stored in a register, which is previously allocated, as shown in Fig. 18 that specifies the content of each file (see Fig. 19).

Once the program has been generated, we can configure the cache architecture model that we want to apply during execution. In this case, a two-level cache will be used with the section for instructions, and the section for data separated at both levels (L1_I + L1_D + L2_I + L2_D). The block size of each cache will be 128 bits for instructions and 64 bits for data. In addition, the number of cache lines will be set at 32 lines, and the number of lines per set will be set at 32 lines to apply a fully associative location policy. In order to apply the architecture replacement policy, in this case, FIFO will be used, as shown in Fig. 20. However, any other type of configuration is valid for the use case.

As can be shown in Fig. 21, during program execution, the module displays every time which instruction is currently being executed. Also,

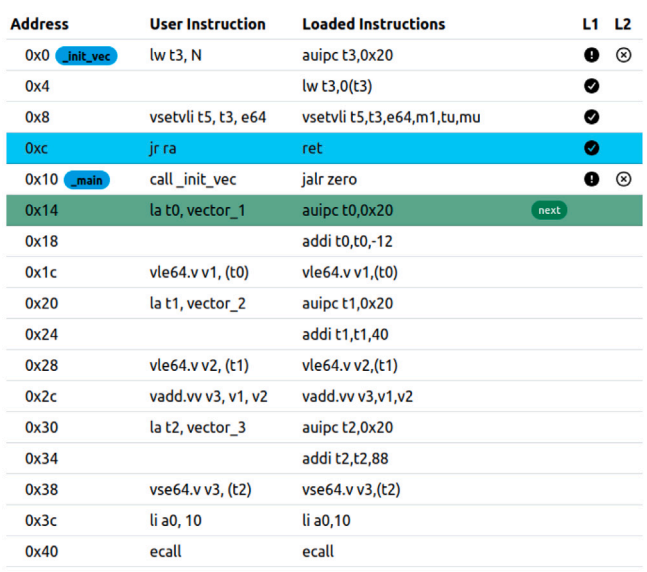

Fig. 21. User interface that displays the execution status of an assembly program.

Fig. 22. User interface that displays the result of the vector operation (*vadd*) stored in destination register (*v3*).

the user can also see in each instruction if there are hits and misses on the cache at the data and instruction levels.

Once execution is completed, the result is observed in the register, in the memory map, and in the cache memory map, allowing verification that the result matches what was expected. The desired result for this use case is that the vector where the result of the operation ($v3$) is stored contains eight 64-bit values, whose content is ten (*0x000000000000000A* in hexadecimal) and that these values are stored in memory, in the space allocated in the data segment. As shown in Fig. 22, the result matches with the result that was expected. This result should also be stored in memory.

Finally, the memory map can be checked to ensure that the result is stored correctly. As can be seen in Fig. 23, the result is stored successfully, and this module displays the value of each position in the vector.

Additionally, the user can check the cache memory map to verify the functionality of the cache memory architecture and the replacement policy applied during execution.

5.2. Use case: Introduce a new instruction defined with Sail

For this use case, a new instruction will be defined using Sail and then compiled using the new compiler before being executed using the

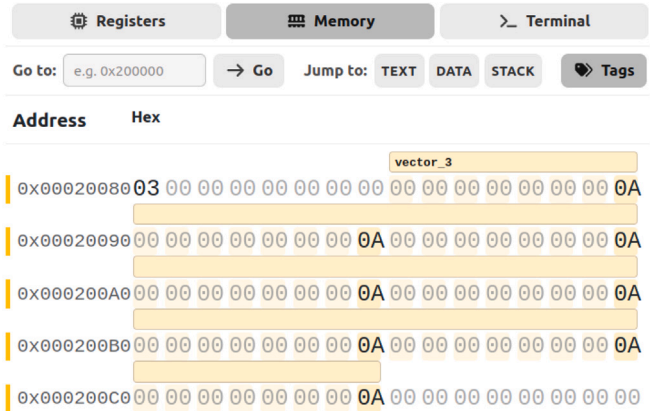

Fig. 23. User interface that displays the result of the operation stored in memory space.

$$rd = rd + rs1$$

Fig. 24. Behavior of new instruction (*myadd*) to be introduced.

```
myadd rd rs1   31..20=0   14..12=0 6..2=0x02 1..0=3
```

Fig. 25. Encoding of the *myadd* new instruction.

new simulator presented previously. The workflow followed for this use case is shown in Fig. 5.

To define a new instruction, the following steps must be followed:

1. Define the structure and the encoding of the instruction.
2. Introduce the definition into the compiler.
3. Define the behavior with Sail and introduce it into the simulator, where it will be executed.

The repository of *RISC-V opcodes*[5] will be used to define the encoding and structure of the instruction. This repository enumerates standard RISC-V instruction opcodes and contains a script to convert the encoding defined in several formats to be applied in other projects, such as, for example, the new compiler introduced into the simulator.

In this case, an instruction will be defined that behaves similarly to the *add* instruction, but with the peculiarity that it only needs two registers. One of them is both a destination register and a source register. Therefore, the operation would be that shown in Fig. 24.

In order to encode a new instruction, the type of instruction must first be identified (e.g., operations with registers, operations with immediate values, conditional operations, etc.). Once this has been established, the encoding can be defined. In this case, it is an operation with registers. Once the opcode field and the two necessary register fields for the encoding instruction have been identified, the rest of the encoding must be marked as zeros at the bit level.

As shown in Fig. 25, the encoding has been defined in the following order: the name of the instruction (*myadd*), its operators (*rd* and *rs1*), the different fields of the instruction that are marked as zeros to fill the 32 bits of the instruction and the encoding of the *opcode*. The *opcode* usually is defined at the last seven bits of the instruction encoding; in this case, the encoding of the operation is *0001011* in binary and *0xB* in hexadecimal.

Once it is defined, the implementation must be checked. To achieve this, a script is run to check whether any other definitions have the same structure and encoding. This notifies the user that their definition

[5] Available at https://github.com/riscv/riscv-opcodes

```
#define MASK_MYADD  0xFFF0707F
#define MATCH_MYADD 0xB
```

Fig. 26. Mask and opcode generated to define the new instruction (*myadd*) in the simulator and compiler.

```
{"myadd", 0, INSN_CLASS_I, "d,s", MATCH_MYADD,
     MASK_MYADD, match_opcode, INSN_ALIAS}
```

Fig. 27. Compiler implementation of the new *myadd* instruction.

```
union clause ast = CREATOR : (regidx, regidx, cop)

mapping encdec_creator : cop <-> bits(7) = {
   CREATOR_ADD <-> 0b0001011
}

mapping clause encdec = CREATOR(rd, rs1, op)
   <-> 0b000000000000 @ rs1 @ 0b000 @ rd @ encdec_creator(op)


mapping creator_mnemonic : cop <-> string = {
   CREATOR_ADD <-> "myadd"
}

mapping clause assembly = CREATOR(rd, rs1, op)
   <-> creator_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
       reg_name(rs1)
```

Fig. 28. Definition, encoding and decoding structure of the new *myadd* instruction.

is not possible. Once this has been done, the script runs a parser to generate the encoding in several formats for introduction into the compiler.

When the parser returns the encoding of the instruction, the implementation of the new instruction continues in both the compiler and the simulator.

To introduce the new instruction, it is necessary to introduce the output generated by the script into the *RISC-V* files of the compiler. The files affected by the compiler[6] are related to the *opcodes* of RISC-V.

As shown in Fig. 26, the *opcode* matches the previous implementation, and the mask was generated to correctly identify the instructions and their structure. This code must be entered into the compiler, and the user must specify whether the instruction is 32-bit, 64-bit or both.

As can be seen in Fig. 27, this line specifies to the compiler the assembly opcode instruction that the user wants to introduce, the variant of the instruction (0 for both variants, 32 for the 32-bit variant only or 64 for the 64-bit variant only), the type of instructions, the operators of the instruction, their encoding (*match* and *mask*), if it should be matched with the opcode to be run and if it is an instruction alias or pseudo-instructions (in this case, it is an instruction alias).

Once all these are integrated into the compiler code, the user can generate their own version of the compiler tool for web environments.

In the simulator part, the user needs to define the behavior of the instruction.

As shown in Fig. 28, the encoding of the instruction, how it is translated when it is decoded and its structure are defined in Sail.

Finally, in Fig. 29, the behavior of the instruction was defined with Sail, and as can be seen, the operation was the same as that in Fig. 24. Once it is implemented, Sail verifies the implementation and generates the simulator in both variants to include the execution of this new instruction.

However, the correct behavior of the architecture can be verified when generating the new components to be used in the simulator by checking its execution in the validation section of the architecture view, as shown in Fig. 30.

[6] More information is available at https://github.com/riscv-collab/riscv-gnu-toolchain.

```
function clause execute CREATOR(rd, rs1, op) = {
    let rd_val = X(rd);
    let rs1_val = X(rs1);
    let result : xlenbits = match op {
        CREATOR_ADD => rd_val + rs1_val
    };
    X(rd) = result;
    RETIRE_SUCCESS
}
```

Fig. 29. Definition of the behavior of the new *myadd* instruction.

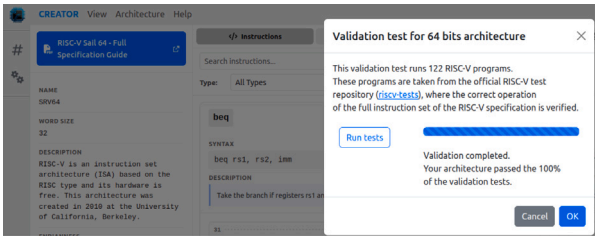

Fig. 30. Architecture and ISA validation process in the web simulator user interface using the validation test suite.

```
.section .bss
    .align 8
tohost: .dword 0

.section .text.init
.globl _main

# Entry point of the program
_main:
    li t0, 10
    li t1, 2
    myadd t0, t1

    jr ra
```

Fig. 31. Assembly program to check the implementation of the new *myadd* instruction.

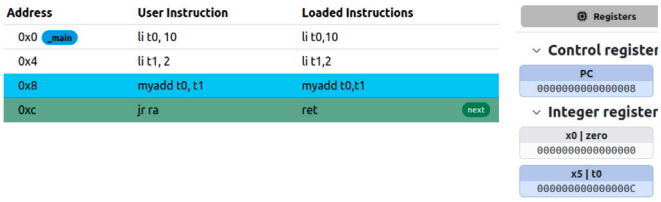

Fig. 32. Result of execution of myadd instruction.

Fig. 31 shows the execution of a program that includes the new instruction.

As shown in Fig. 32, the result of executing the instruction is as expected, with the addition of the two register values and the result being stored in register t0.

6. Conclusions and future work

This article introduces a new web simulator that employs Sail as its instruction specification language. The simulator provides a new tool for simulating and encoding programs in assembly code for the 32-bit and 64-bit variants of the RISC-V architecture, incorporating the full set of instructions defined in the RISC-V standard.

This new version of CREATOR opens the door to the development of programs for industrial use and research-level instruction definition thanks to Sail, which is a specification language that allows us to explore much more of the architecture definition. Unlike other architecture specification tools, Sail can verify the definition, behavior and semantics, which can potentially generate undefined behaviors. For example, this has occurred in previous processor development situations, as in the case of Intel [28].

The WebAssembly compilation tool stands out in the development process. This tool enables the user to generate executable applications in browsers with a performance comparable to that of native applications. The main feature is that it is not necessary to modify the program code to adapt it to a web environment, making the development process and its integration into CREATOR easier. However, modifications have to be made during development because WebAssembly is not capable of adapting all GCC functionalities to a web environment. In particular, those functionalities that depend on the operating system cannot be adapted by this tool. Even so, WebAssembly has been adapted so that the behavior of both the simulator and the toolchain is the same in both web environments and native environments. Finally, it is also worth noting the recent update that enables 64-bit applications to be run with WebAssembly in browsers [33,34], which facilitated the integration of a 64-bit simulator into the web simulator.

The future work expected from this tool includes, among other things, the expansion of replacement cache policies to be applied during the simulation and testing of the execution of a minimal operating system that can be interacted with through the web simulator. It will also involve the integration of different modules designed and tested with Sail, thereby expanding the use cases of the web simulator and opening the door to industrial jobs using RISC-V in web environments while minimizing costs.

CRediT authorship contribution statement

Juan Carlos Cano-Resa: Writing – review & editing, Writing – original draft, Validation, Software, Investigation. **Felix Garcia-Carballeira:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Diego Camarmas-Alonso:** Writing – review & editing, Writing – original draft, Validation, Software, Investigation. **Alejandro Calderon-Mateos:** Writing – review & editing, Writing – original draft, Supervision, Software, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is a part of the project “Integrated development environment for education and research in RISC-V processors,” Ref. PDC2023-145832-I00, funded by MICIU/ AEI/10.13039/501100011033 and by European Union ”NextGenerationEU/PRTR”. This work has also been funded by APC: Universidad Carlos III de Madrid (Agreement CRUE-Madroño 2026).

Data availability

Research Link Provided.

References

[1] J. Saidova, RISC-V architecture and its role in the near future, J. Adv. Sci. Res. (ISSN: 0976-9595) 5 (9) (2024).

[2] S. Cieslak, A. Oleksiak, K. Marcinek, W.A. Pleskacz, Retargeting the MIPS-II CPU core to the RISC-V architecture, in: 2019 MIXDES - 26th International Conference on Mixed Design of Integrated Circuits and Systems, 2019, pp. 261–264, http://dx.doi.org/10.23919/MIXDES.2019.8787018.

[3] QEMU, QEMU - A generic and open source machine emulator and virtualizer, 2025, URL https://www.qemu.org/docs/master/about/index.html. (Accessed on 6 March 2025) [Online].

[4] RISC-V Software, Spike RISC-V ISA Simulator, 2025, URL https://github.com/riscv-software-src/riscv-isa-sim. (Accessed on 6 MArch 2025) [Online].

[5] D. Camarmas-Alonso, F. Garcia-Carballeira, A. Calderon-Mateos, E. del Pozo-Puñal, CREATOR: An educational integrated development environment for RISC-V programming, IEEE Access 12 (2024) 127702–127717, http://dx.doi.org/10.1109/ACCESS.2024.3406935.

[6] MIPS, MIPS - A leading provider of RISC-V and MIPS IP cores, 2025, URL https://mips.com/. (Accessed 6 March 2025) [Online].

[7] RISC-V International, RISC-V International - The free and open RISC architecture, 2025, URL https://riscv.org/. (Accessed 6 March 2025) [Online].

[8] RISC-V Foundation, The RISC-V Instruction Set Manual Volume I, 2024, URL https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link. (Accessed 6 March 2025) [Online].

[9] RISC-V Foundation, The RISC-V Instruction Set Manual Volume II, 2024, URL https://drive.google.com/file/d/17GeetSnT5wW3xNuAHI95-SI1gPGd5sJ_/view.

[10] K.E. Gray, P. Sewell, C. Pulte, S. Flur, R. Norton-Wright, The Sail instruction-set semantics specification language, Tech. rep., Cambridge University, 2017.

[11] J. Bartlett, Debugging with GDB, in: Learn To Program with Assembly: Foundational Learning for New Programmers, A Press, Berkeley, CA, 2021, pp. 265–269, http://dx.doi.org/10.1007/978-1-4842-7437-8_22, URL https://doi.org/10.1007/978-1-4842-7437-8_22.

[12] Antmicro, Renode. https://github.com/renode/renode/tree/master.

[13] K.V. Pete Sanderson, RARS – RISC-V Assembler and Runtime Simulator. https://github.com/TheThirdOne/rars.

[14] M.B. Petersen, Ripes: A visual computer architecture simulator, in: 2021 ACM/IEEE Workshop on Computer Architecture Education, WCAE, IEEE, 2021, pp. 1–8.

[15] K. Vakil, Venus. https://github.com/kvakil/venus.

[16] C. Simulator, CREATOR - RISC-V Simulator, 2025, URL https://creatorsim.github.io/. (Accessed 6 March 2025) [Online].

[17] RISC-V, RISC-V Proxy Kernel. (accessed on 6 march 2025) [online], 2025, URL https://github.com/riscv-software-src/riscv-pk.

[18] P. Sanderson, MARS (official) MIPS Assembler and Runtime Simulator. https://github.com/dpetersanderson/MARS.

[19] Czech Technical University, QtRvSim. https://github.com/cvut/qtrvsim.

[20] S. Macke, B. Burns, G. Braad, N. Gupta, L. Angrave, jor1k. https://github.com/s-macke/jor1k/.

[21] Takahiro, F. Uhlig, A. Bhosale, Kurun, riscv-rust. https://github.com/takahirox/riscv-rust.

[22] V.M. de Morais Costa, Vulcan. https://github.com/vmmc2/Vulcan.

[23] Boostrap, 2026, URL https://getbootstrap.com. (Accessed 7 March 2026) [Online].

[24] Vue, 2026, URL https://vuejs.or. (Accessed 7 March 2026) [Online].

[25] World Wide Web Consortium, WebAssembly, 2025, URL https://webassembly.org/docs/faq/. (Accessed 7 March 2025) [Online].

[26] B. Spies, M. Mock, An evaluation of WebAssembly in non-web environments, in: 2021 XLVII Latin American Computing Conference, CLEI, 2021, pp. 1–10, http://dx.doi.org/10.1109/CLEI53233.2021.9640153.

[27] Emscripten, Emscripten – Complete compiler toolchain to WebAssembly, using LLVM, with a special focus on speed, size, and the Web platform. https://emscripten.org/index.html.

[28] J. Harrison, Verification: Industrial Applications Notes to accompany lectures at 2003 Marktoberdorf Summer School, 2003,

[29] Isabelle, Isabelle/HOL Overview, 2025, URL https://isabelle.in.tum.de/overview.html. (Accessed on 7 March 2025) [Online].

[30] Australian National University, Chalmers University of Technology, HOL4, 2025, URL https://hol-theorem-prover.org/. (Accessed 7 March 2025) [Online]).

[31] C.P.-M. Thierry Coquand, Coq, 2025, URL https://coq.inria.fr/. (Accessed 7 March 2025) [Online].

[32] J. De Macedo, R. Abreu, R. Pereira, J. Saraiva, WebAssembly versus JavaScript: Energy and runtime performance, in: 2022 International Conference on ICT for Sustainability (ICT4S), IEEE, 2022, pp. 24–34.

[33] World Wide Web Consortium, WebAssembly 64 bits Compiler, 2025, URL https://github.com/WebAssembly/memory64. (Accessed on 7 March 2025) [Online].

[34] Memory64, WebAssembly 64 bits proposal, 2025, URL https://webassembly.org/features/. (Accessed 7 March 2025) [Online].